Tokenization

Multilevel Attention - represent current token is looking for other token

Attention Score - how relevant one token is to another

Query
↓
Key
↓
Value

$$Score = \frac{Q \times K^T}{\sqrt{d_k}}$$

Encoder - converge one data from one form to another

Marking - prevent the model from attending to certain token that it should not use

Diamension

softmax - list of no (0, 1)

$$\frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

$e^1 = 2.718$

$e^2 = 7.389$

$e^3 = 20.086$

sum = 30.193

$1 = \frac{2.718}{30.193} = 0.090$

$2 = \frac{7.389}{30.193} = 0.245$

$3 = \frac{20.086}{30.193} = 0.665$

$P = 1$

Retriver

GPT Arch - (Generative Pre-trained Transformer)

Decoder only Transformer

[Input - tokenizer - Token Embeddings -> P.E -> Transformer Decoder block -> linear - softmax -> next token prediction]

Limitation
- -> Halluniaction
- -> Knowledge limitation

BERT (Bidirectional Encoder Representations from Transformers)

Designed mainly for understanding language

- -> Masked Lang Model
- -> Next sentence predection

parameters - inferred values that a neural network learns

Tokens - smallest word possible llm process tokens

Stage of LLM

Fine tuning → Allignment

Biggest Learning phase it uses self superised Learning

training on small dataset

make the model behave

training on smaller (or) specialized dataset

Types

- full tuning
- Parameter efficient
- LORA
- QoRA
- supervised
- Instruction fine tuning
- Transfer Learning

Key concepts

Temp - control randomness

Top K - sampling - only consider top k

Top P - chose token until probability reach p

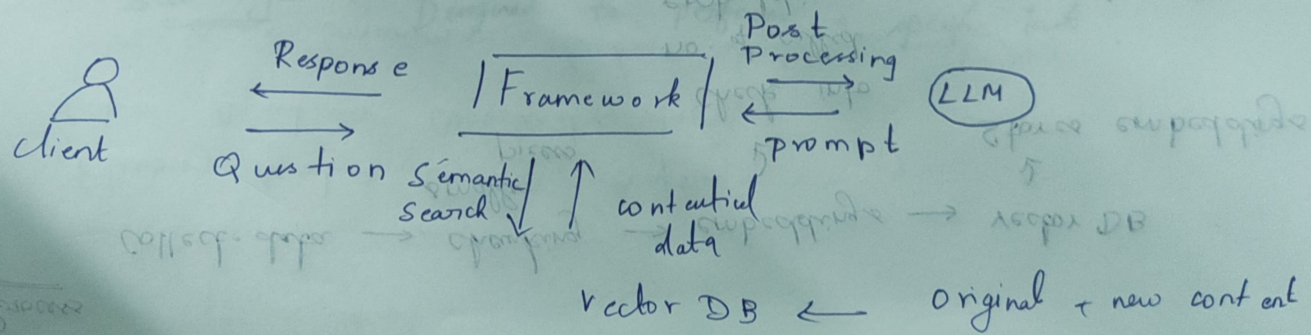
Hallucination - Info that sound correct but wrong

RAG (Retrieval Augmented Generation)

Tech that improves LLM by giving them access to external knowledge sources

RAG = LLM + Search engine + Database

Architecture



Process

Collect data → chunking → embeddings → vector DB

smaller pieces

↓ Check into no. of called embeddings → meaning of text

↓ Stores embeddings

R + G

Query embedding → Similar search → Augmentation → generation

Types of Retrival

Dense

Sparse

Hybrid

Create an AI agent

→ Define goal
→ Choose an LLM
→ Create the state
→ Create Tools
→ Create Agent workflow
→ Add memory
→ Add Decision Making
→ Implement using framework

Multi-agent AI systems

→ a complex task divided among multiple ag

Super Agent - Main controling agent

Sub Agent - Designed to perform a specific task

Ex Draft a email

Goal → Write a email

Planning Agent → Creates drafting strategy

Super Agent → Coordinates tasks

Tools → Grammar checker, calendar, Email API

State → Stores draft progress

Feedback Loop → Improves email quantity